
title: Deploy Express Backend on Vercel **subtitle:** Reusable guide for hosting Node.js APIs on another project

Deploy Express Backend on Vercel

Purpose: Step-by-step guide to host an **Express + MongoDB** (or similar) backend on Vercel as a serverless API, configure environment variables, connect a Next.js (or other) frontend, and avoid common deployment issues.

Estimated time: 45–60 minutes (first deployment)

Table of Contents

1. [How Vercel Runs Your Backend](#)
 2. [Prerequisites](#)
 3. [Required Project Structure](#)
 4. [Create vercel.json](#)
 5. [Create the Serverless Entry Point](#)
 6. [Adapt server.js for Vercel](#)
 7. [Database — MongoDB Atlas](#)
 8. [Environment Variables on Vercel](#)
 9. [Deploy from GitHub](#)
 10. [Connect Your Frontend](#)
 11. [CORS and FRONTEND_URL](#)
 12. [File Uploads on Vercel \(Use S3\)](#)
 13. [Limits and Production Notes](#)
 14. [Troubleshooting](#)
 15. [Checklist for New Projects](#)
-

1. How Vercel Runs Your Backend

Vercel is optimized for **serverless functions**, not a long-running `node server.js` process.

- Each request invokes a function (cold start possible).
- Your Express `app` is exported and wrapped by a single handler at `api/index.mjs` (or `api/index.js`).

- `vercel.json` rewrites all routes to that handler so `/api/health`, `/api/auth/login`, etc. work.

Do not rely on `app.listen()` on Vercel — Vercel sets `VERCEL=1` and invokes the exported handler instead.

2. Prerequisites

- GitHub (or GitLab/Bitbucket) repository with your backend code
 - [Vercel account](#) linked to Git
 - **MongoDB Atlas** cluster (or another cloud database reachable from the internet)
 - Domain or Vercel default URL for frontend and backend (can be two separate Vercel projects)
-

3. Required Project Structure

Example layout (backend as its own repo or monorepo subfolder):

```
backend/
├── api/
│   └── index.mjs           # Vercel serverless entry
├── src/
│   ├── app.js             # Express app (routes, middleware)
│   ├── server.js          # Local dev only: listen + export app
│   ├── config/
│   ├── controllers/
│   ├── routes/
│   └── ...
├── package.json
├── vercel.json
└── .env.example
```

Root Directory: If the repo is a monorepo, set Vercel **Root Directory** to `backend` when importing the project.

4. Create `vercel.json`

Place in the backend root (same folder as `package.json`):

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": null,
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/api"
    }
  ]
}
```

What this does: Every incoming path is handled by the function at `api/index.mjs` (Vercel maps `/api` to the `api/` folder).

5. Create the Serverless Entry Point

File: `backend/api/index.mjs`

```
import dotenv from "dotenv";

dotenv.config();

import app from "../src/app.js";

export default function handler(req, res) {
  return app(req, res);
}
```

- Load `dotenv` so local `vercel dev` can read `.env`.
 - On production Vercel, variables come from the Vercel dashboard (not from committed `.env`).
-

6. Adapt server.js for Vercel

File: `backend/src/server.js`

```
import dotenv from "dotenv";
import app from "./app.js";
import { ensureAppInitialized } from "./config/init.js";

dotenv.config();

const PORT = process.env.PORT || 8081;

async function startLocalServer() {
  await ensureAppInitialized();
  app.listen(PORT, () => {
    console.log(`[server] running on http://localhost:${PORT}`);
  });
}

// Only start HTTP server when NOT on Vercel
if (process.env.VERCEL !== "1") {
  startLocalServer();
}

export default app;
```

Important:

- Export `app` for the serverless handler.
- Gate `app.listen()` behind `VERCEL !== "1"`.
- Run DB connection / seed logic in middleware or `ensureAppInitialized()` per request (with caching) so cold starts connect once.

Disable seed on Vercel production:

```
function shouldRunSeed() {
  if (process.env.NODE_ENV === "production") return false;
  if (process.env.VERCEL === "1") return false;
  return process.env.RUN_SEED_ON_BOOT === "true";
}
```

7. Database — MongoDB Atlas

1. <https://cloud.mongodb.com/> → create cluster (free tier available).
2. **Database Access** → create user + password.
3. **Network Access** → **Add IP Address** → `0.0.0.0/0` (allow from anywhere) — required because Vercel serverless IPs change. For stricter security, use Atlas VPC peering or a fixed-IP proxy (advanced).

4. **Connect** → Drivers → copy connection string:

```
mongodb+srv://USER:PASSWORD@cluster0.xxxxx.mongodb.net/dbname?retryWrites=true&w=ma
```

Set on Vercel as:

```
MONGODB_URI=mongodb+srv://...
```

8. Environment Variables on Vercel

Vercel Dashboard → your **backend** project → **Settings** → **Environment Variables**.

Add at minimum:

Variable	Example	Notes
MONGODB_URI	mongodb+srv://...	Atlas connection string
JWT_SECRET	long random string	Auth signing
FRONTEND_URL	https://my-app.vercel.app,http://localhost:3000	Comma-separated CORS origins
NODE_ENV	production	Optional; Vercel sets runtime
AWS_ACCESS_KEY_ID	...	If using S3 uploads
AWS_SECRET_ACCESS_KEY	...	If using S3
AWS_REGION	ap-south-1	S3 region
AWS_BUCKET_NAME	my-bucket	S3 bucket
ADMIN_ALERT_EMAIL	admin@example.com	If using email
SMTP_HOST , SMTP_USER , SMTP_PASS	...	If using Nodemailer

Scopes: Enable for **Production**, **Preview**, and **Development** as needed.

After changing variables → **Redeploy** (Deployments → : → Redeploy).

Never commit `.env` with real secrets to Git.

9. Deploy from GitHub

1. Push backend code to GitHub.
2. Vercel → **Add New Project** → Import repository.
3. **Root Directory:** `backend` (if monorepo).
4. **Framework Preset:** Other (not Next.js for backend-only).
5. **Build Command:** leave empty or `npm install` if needed.
6. **Output Directory:** leave default (not used for serverless API).
7. Add environment variables (section 8).
8. **Deploy.**

Your API base URL will look like:

```
https://your-backend-project.vercel.app
```

Routes:

- `https://your-backend-project.vercel.app/api/health`
- `https://your-backend-project.vercel.app/api/auth/login`

(Assuming Express mounts routes under `/api` in `app.js`.)

10. Connect Your Frontend

For a **Next.js** frontend on Vercel, set:

```
NEXT_PUBLIC_API_BASE=https://your-backend-project.vercel.app/api
```

Rules:

- Must be the **full HTTPS URL**.
- Must end with `/api` if your Express app uses `app.use("/api", ...)`.

Redeploy frontend after setting the variable.

Local frontend + production API:

```
NEXT_PUBLIC_API_BASE=https://your-backend-project.vercel.app/api
```

Local frontend + local API:

```
NEXT_PUBLIC_API_BASE=http://localhost:5000/api
```

11. CORS and FRONTEND_URL

Backend should allow origins from `FRONTEND_URL` (comma-separated):

```
FRONTEND_URL=https://my-frontend.vercel.app,https://www.mydomain.com,http://localhost
```

- No trailing slash required (normalize in code).
- Every Vercel preview URL for frontend: add to list or use a single production URL for admin only.
- If browser shows **CORS error**, the request `Origin` is not in this list — add it and redeploy backend.

12. File Uploads on Vercel (Use S3)

Vercel serverless has **no persistent disk**. Do not save uploads to `./uploads` on the server.

Use:

- **Amazon S3** (recommended) with presigned URLs or API upload through multer → memory → S3
- Or Cloudinary / similar

Configure all `AWS_*` variables on the backend Vercel project. See the companion PDF: **S3 Bucket Setup Guide**.

13. Limits and Production Notes

Topic	Vercel serverless
Request body size	~4.5 MB (Hobby); larger on Pro
Execution timeout	10s Hobby / up to 60s+ Pro

Topic	Vercel serverless
Cold starts	First request after idle may be slower
WebSockets	Not ideal on standard serverless
Long-running jobs	Use external queue (SQS, Inngest, etc.)
Cron	Use Vercel Cron → hit an API route

For heavy traffic or WebSockets, consider Railway, Render, Fly.io, or AWS ECS — Vercel is excellent for REST APIs and admin backends.

14. Troubleshooting

Issue	Cause	Fix
404 on all routes	Missing <code>vercel.json</code> rewrites or wrong root directory	Add rewrites; set Root Directory to <code>backend</code>
<code>FUNCTION_INVOCATION_FAILED</code>	Crash on import (syntax, missing module)	Check deployment logs; run <code>npm install</code> locally
DB connection timeout	Atlas IP not allowed	Allow <code>0.0.0.0/0</code> or fix network rules
CORS blocked	<code>FRONTEND_URL</code> missing origin	Add exact frontend URL; redeploy
Env undefined in API	Vars not set on Vercel	Add in dashboard; redeploy
Upload fails	Disk not persistent	Use S3
<code>app.listen</code> EADDRINUSE locally	Port in use	Change <code>PORT</code> in <code>.env</code>
Works locally, 500 on Vercel	<code>VERCEL=1</code> code path / init error	Read Functions logs in Vercel dashboard
Frontend "NEXT_PUBLIC_API_BASE required"	Missing frontend env	Set on frontend Vercel project

View logs: Vercel → Project → **Deployments** → select deployment → **Functions / Runtime Logs**.

Local Vercel simulation:

```
npm i -g vercel
cd backend
vercel dev
```

15. Checklist for New Projects

- ☐ `api/index.mjs` exports handler wrapping Express `app`
- ☐ `vercel.json` rewrites `/(.*)` → `/api`
- ☐ `server.js` does not call `listen()` when `VERCEL === "1"`
- ☐ `app` exported as default from `server.js`
- ☐ MongoDB Atlas cluster + `MONGODB_URI` on Vercel
- ☐ `JWT_SECRET` set (strong, unique)
- ☐ `FRONTEND_URL` includes all frontend origins
- ☐ S3 (or cloud storage) configured for uploads
- ☐ `.env` not committed; `.env.example` documented
- ☐ Backend deployed; `/api/health` returns OK
- ☐ Frontend `NEXT_PUBLIC_API_BASE` points to `https://...vercel.app/api`
- ☐ CORS tested from browser (login, CRUD, upload)
- ☐ Redeploy after any env change

Quick Reference — Minimal Files

`vercel.json`

```
{
  "framework": null,
  "rewrites": [{ "source": "/(.*)", "destination": "/api" }]
}
```

`api/index.mjs`

```
import dotenv from "dotenv";
dotenv.config();
import app from "../src/app.js";
export default (req, res) => app(req, res);
```

Vercel env (minimum)

```
MONGODB_URI=...  
JWT_SECRET=...  
FRONTEND_URL=https://your-frontend.vercel.app
```

Document version: 1.0

Applies to: Express (ESM) + Vercel Serverless + MongoDB Atlas + optional S3